

UNIVERSIDAD NACIONAL DE LOJA



1859



Universidad
Nacional
de Loja

Carrera: Ingeniería en Computación

Asignatura: Teoría de la Programación

Docente: Lisete Geoconda López Faican

Unidad: 1

Tema: Exploración y uso de herramientas digitales para pseudocódigo y diagramas de flujo

Nombre: Matias Joel Ortega Correa

Fecha: 03/05/2026

• CONTENIDOS

Algoritmo

Un algoritmo es un conjunto ordenado, finito y definido de instrucciones o reglas paso a paso que permiten solucionar un problema, realizar un cómputo o procesar datos. Funciona recibiendo datos de entrada (input), procesándolos mediante reglas lógicas y produciendo un resultado final (output). Son la base de la programación y la tecnología moderna.

Realice un algoritmo llamado “calculodegalones” que tiene como propósito convertir la producción diaria ingresada en litros a su equivalente en galones y, a partir de ello, calcular el monto total a cobrar según un precio por galón. Primero se definen las variables necesarias para almacenar los datos de entrada y los resultados, luego se solicita al usuario que ingrese la cantidad de litros producidos y el precio por galón. Posteriormente, en el proceso, se realiza la conversión de litros a galones utilizando el factor 3.785 y se calcula el total a pagar multiplicando los galones obtenidos por el precio ingresado. Finalmente, el algoritmo muestra en pantalla los resultados, es decir, la cantidad equivalente en galones y el total a cobrar, permitiendo así obtener de forma sencilla tanto la conversión de unidades como el cálculo económico correspondiente.

Pseudocódigo

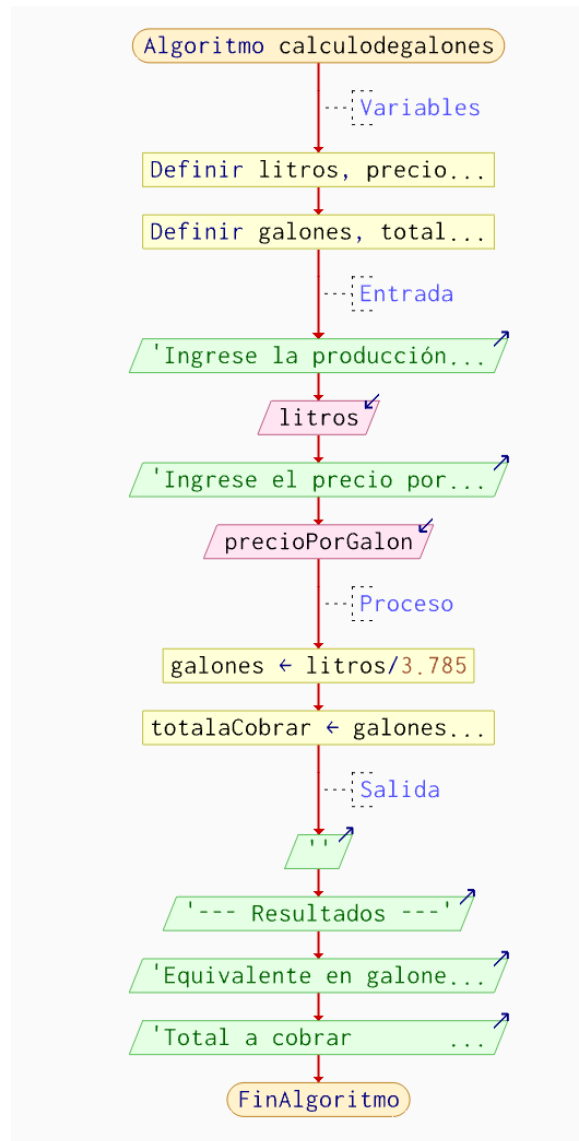
El pseudocódigo es una forma de representar algoritmos y la lógica de programación utilizando una mezcla de lenguaje natural (español) y estructuras de código, sin las reglas sintácticas estrictas de un lenguaje real. Sirve como borrador para planificar programas, enfocándose en el flujo lógico antes de la codificación

En este caso presentamos el algoritmo de “calculodegalones”:

```
1  Algoritmo calculodegalones
2
3      //Variables
4      Definir litros, precioPorGalon Como Real
5      Definir galones, totalaCobrar Como Real
6
7      //Entrada
8      Escribir "Ingrese la producción del día (en litros):"
9      Leer litros
10
11     Escribir "Ingrese el precio por galón ($):"
12     Leer precioPorGalon
13
14     //Proceso
15     galones ← litros / 3.785
16     totalaCobrar ← galones * precioPorGalon
17
18     //Salida
19     Escribir ""
20     Escribir "--- Resultados ---"
21     Escribir "Equivalente en galones: ", galones, " galones"
22     Escribir "Total a cobrar          : $", totalaCobrar
23
24
25 FinAlgoritmo
```

Diagrama de flujo

Un diagrama de flujo (o flujograma) es una representación gráfica de un proceso, algoritmo o sistema, que utiliza símbolos estandarizados conectados por flechas para secuenciar pasos. Permite visualizar de forma clara y ordenada acciones, decisiones y el flujo de información desde el inicio hasta el final de una tarea.



Prueba de escritorio

Una prueba de escritorio es la simulación manual y paso a paso de un algoritmo, pseudocódigo o código, utilizando papel y lápiz (o una tabla) para rastrear los valores de las variables y verificar la lógica sin ejecutarlo en la computadora. Es fundamental para detectar errores tempranos, validar la funcionalidad y entender el flujo del programa.

Paso	litros	precioPorGalon	galones	totalaCobrar
Entrada	15	5		
Conversión	15	5	3.96	
Cálculo	15	5	3.96	19.80

Paso	litros	precioPorGalon	galones	totalaCobrar
Entrada	50	3.2		
Conversión	50	3.2	13.21	
Cálculo	50	3.2	13.21	42.27

Lenguajes de programación

Los lenguajes de programación son sistemas formales utilizados para dar instrucciones a las computadoras, divididos principalmente en alto nivel (ceranos al humano) y bajo nivel (ceranos al hardware). Los más populares y utilizados incluyen Python (versátil), JavaScript (desarrollo web), Java (empresarial), C# (videojuegos) y C/C++ (rendimiento).

En clase usamos el lenguaje C en Visual Studio porque te permite entender las bases fundamentales de la programación, como el manejo de memoria, estructuras de control y lógica computacional de forma clara y directa. C es un lenguaje de bajo nivel comparado con otros, así que te obliga a comprender cómo funciona realmente el computador, lo cual fortalece tus habilidades para aprender otros lenguajes más fácilmente. Además, Visual Studio ofrece herramientas como depuración paso a paso y autocompletado que facilitan el aprendizaje, ayudándote a detectar errores y mejorar tu código de manera más rápida y profesional.

Mi pseudocódigo transformado en lenguaje C:

```

1  #include <stdio.h>
2
3  int main() {
4      float litros, precioPorGalon;
5      float galones, totalACobrar;
6
7      // Entrada
8      printf("Ingrese la produccion del dia (en litros): ");
9      scanf("%f", &litros);
10
11     printf("Ingrese el precio por galon ($): ");
12     scanf("%f", &precioPorGalon);
13
14     // Proceso
15     galones = litros / 3.785;
16     totalACobrar = galones * precioPorGalon;
17
18     // Salida
19     printf("\n--- Resultados ---\n");
20     printf("Equivalente en galones: %.2f\n", galones);
21     printf("Total a cobrar: $ %.2f\n", totalACobrar);
22
23     return 0;
24 }

```

Programación por bloques

La programación por bloques es un método visual para crear software arrastrando y encajando piezas gráficas (bloques) que representan comandos, eliminando la necesidad de escribir sintaxis compleja. Es ideal para principiantes y niños, ya que permite aprender lógica de programación secuencial de forma intuitiva, lúdica y sin errores de escritura.

- **EJERCICIO CON ESTRUCTURA SECUENCIAL (LENGUAJE C EN VISUAL STUDIO)**

- **Planteamiento del problema**

Desarrollar un programa en lenguaje C que permita ingresar la base y la altura de un rectángulo, y calcule su área.

- **Análisis del problema**

Datos de entrada: Base (b), Altura (h).

Proceso: Área = base × altura

Salida: Área del rectángulo

- **Diseño del algoritmo**

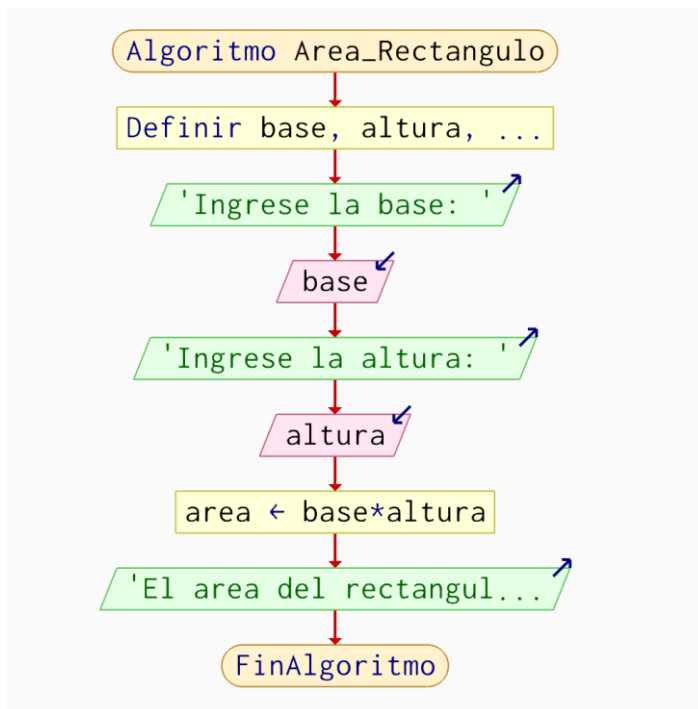
Pseudocódigo:

```

1  Algoritmo Area_Rectangulo
2
3      Definir base, altura, area Como Real
4
5      Escribir "Ingrese la base: "
6      Leer base
7
8      Escribir "Ingrese la altura: "
9      Leer altura
10
11     area ← base * altura
12
13     Escribir "El area del rectangulo es: ", area
14
15 FinAlgoritmo

```

Diagrama de flujo:



➤ Codificación (C en Visual Studio)

```

1  #include <stdio.h>
2
3  int main() {
4      float litros, precioPorGalon;
5      float galones, totalACobrar;
6
7      // Entrada
8      printf("Ingrese la produccion del dia (en litros): ");
9      scanf("%f", &litros);
10
11     printf("Ingrese el precio por galon ($): ");
12     scanf("%f", &precioPorGalon);
13
14     // Proceso
15     galones = litros / 3.785;
16     totalACobrar = galones * precioPorGalon;
17
18     // Salida
19     printf("\n--- Resultados ---\n");
20     printf("Equivalente en galones: %.2f\n", galones);
21     printf("Total a cobrar: $ %.2f\n", totalACobrar);
22
23     return 0;
24 }
25

```

➤ Validación (Prueba de escritorio)

Base	Altura	Cálculo	Área
5	3	5×3	15
2.5	4	2.5×4	10

➤ Principales dificultades de los contenidos.

Durante el desarrollo de este ejercicio en C, una de las principales dificultades fue comprender cómo funcionan las variables, ya que es necesario entender que cada variable almacena un tipo de dato específico y debe ser declarada correctamente antes de usarse. También se presentó cierta dificultad al utilizar la función **scanf**, especialmente en el uso del símbolo **&**, el cual es obligatorio para indicar la dirección de memoria donde se guardará el valor ingresado por el usuario. Finalmente, otro aspecto importante fue diferenciar entre los tipos de datos como **int** y **float**, ya que elegir el tipo correcto es clave para evitar errores en los cálculos, sobre todo cuando se trabajan con números decimales.

➤ Reflexión crítica en la aplicación

La estructura secuencial es considerada la más sencilla dentro de la programación porque las instrucciones se ejecutan una tras otra en un orden establecido, sin tomar decisiones ni repetir

procesos. Este tipo de ejercicios es muy útil para desarrollar la lógica básica de programación, ya que permite enfocarse en la correcta comprensión de las variables, operaciones y entrada/salida de datos. Además, sirve como base fundamental antes de avanzar hacia estructuras más complejas como las condicionales y los ciclos, que requieren un mayor nivel de razonamiento y control del flujo del programa.